

Thinking With AI: Students' Epistemic Practices in Open-Ended Problem Solving

Maurison Agba

Department of Engineering Education

University at Buffalo

Buffalo, USA

maurison@buffalo.edu

Olivia Schupbach

Department of Biomedical Engineering

University of Buffalo

Buffalo, USA

oischupb@buffalo.edu

Courtney Faber

Department of Engineering Education

University at Buffalo

Buffalo, USA

cfaber@buffalo.edu

Introduction

Computer game design projects have been used to strengthen students' problem-solving skills and provide opportunities for them to apply knowledge in real-world contexts [1]. This pedagogical approach has been shown to improve students' participation and learning outcomes in introductory software engineering courses [2]. However, using this instructional method for beginners in software-related courses is challenging due to learners' limited foundational knowledge in areas such as computer programming [3]. Studies have applied Appreciative Inquiry approach, in which students are provided with structured guidance, to computer game design projects to enhance their creative performance [4]. Recently, generative artificial intelligence (GenAI), such as ChatGPT, has been used to support students in completing computer game creation projects (e.g., [5]). While this tool offers significant benefits by providing students with access to essential domain-specific knowledge, facilitating brainstorming, and supporting effective summarization and elaborations that aid problem-

solving, it also has inherent limitations, such as generating incoherent game stories [5] and hallucinations [6]. Consequently, students must critically assess GenAI-generated content before incorporating it into their solution strategies. Students' ability to critically evaluate GenAI answers may be shaped by their beliefs about knowledge and knowing (epistemic beliefs). For example, students may treat GenAI as an authority and incorporate unworkable ideas, or they may critically evaluate its outputs to identify feasible solutions. This tension requires them to generate, evaluate, and justify knowledge claims—a process referred to as epistemic practices [7]. Epistemic practices include, for example, “noticing patterns in data, making observations, and examining the functionality of components of designed objects” [8, p. 403].

In this study, we investigated the ways students used GenAI and the epistemic practices they enacted through this use on an open-ended programming project in an introductory programming course for engineers. Our study was guided by the research questions: In what ways do students use GenAI when solving an open-ended computer programming problem in a second-year introductory programming course, and what epistemic practices do they enact through this use?

Background studies

In our review of the literature, we did not identify any studies that have directly examined students' epistemic practices when using GenAI to solve open-ended computer programming problems. However, scholars are studying how GenAI is being used for game development, both in professional [9], [10] and pedagogical settings [11].

Theoretical framework

We draw on the scholarship of epistemic cognition (EC), defined as “the set of mental processes that involve the development and employment of one's conception of knowledge and knowing” [12, p. 20]. EC is often activated when individuals are faced with open-ended problems [13]. There are various models of EC, all of which agree that individuals can have naïve or sophisticated epistemic cognition. The pioneering stage model conceptualizes EC as progressing through developmental stages from simplistic to the most sophisticated form [14], [15], [16]. At the naïve level, knowledge is seen as absolute, fixed, certain, and handed down by authorities [15]. In contrast, sophisticated EC regards knowledge as constructed and justifiable based on the most reasonable evidence [14]. The belief model of EC treats knowledge as an independent dimension of belief, with each dimension existing on its own naïve-to-sophisticated continuum [17], [18]. Hofer et al. [18] theorized two dimensions of epistemic belief: beliefs about the nature of knowledge and the beliefs about knowing. Unlike stage models, they argue that an individual can hold sophisticated epistemic belief in one dimension while remaining naïve in another. For example, one can view knowledge as complex while also believing that knowledge can be given by authorities. The resource model argues, contrary to the belief model, that an individual can exhibit both naïve and sophisticated EC within the same dimension of belief, depending on context [19]. From this background, it can be inferred that students' inquiry, sharing, and use of information from GenAI will depend on their EC.

Scholarships on EC is increasingly focusing on epistemic practices [8], [20], [21]. Accordingly, we adopted the Aims, Ideals, and Reliable Processes (AIR) model of EC [22] to guide our definition of epistemic practices. This model offers a framework for investigating and interpreting actions within situated contexts. The epistemic aim of the AIR model refers to an individual's set goals that are related to knowledge. For example, these goals may include examining an X ray report to detect an anomaly or measuring the length of a frame to determine whether it can fit into a piece of machinery. Epistemic ideals are standards for evaluating the attainment of epistemic aim. For example, presence of supporting evidence. Reliable epistemic processes are set of methods individuals use to evaluate or accomplish an epistemic aim, for example, confirming the reading of an X-ray from an independent source. From this framework, we developed our definition of epistemic practices. In addition to reviewing models of EC, we examine how scholars categorize engineering epistemic practices.

Simarro and Couso [23] theorized that the epistemic practices in engineering include defining and delimiting problems, analyzing and interpreting data to identify points for improvement, identifying and/or developing multiple solutions and selecting the optimal one, and materializing the solution. Similarly, Cunningham and Kelly's [24] put forward sixteen epistemic practices, organized into four main categories: engineering in social contexts; uses of data and evidence to make decisions; tools and strategies for problem solving; and finding solutions through creativity and innovation. While these practices provide valuable insight into the nature of engineering epistemic practices, they do not address how such practices may be transformed by tools like GenAI.

The use of generative AI in game development

Professional game developers use GenAI during ideation, brainstorming, asset creation, programming, and building game stories [9], [25]. For example, they use tools such as ChatGPT for coding, thereby accelerating the developmental process, as well as for conceptualizing ideas and narratives [26]. The quality and correctness of GenAI's outputs depend significantly on how effectively it is prompted. Even when explicit prompting is used, with users clearly specifying constraints, rules, and expectations, the outputs do not always meet established game standards and often require manual refinement [25], [27], [28]. Some game art concept creators have expressed challenges integrating GenAI into their workflow due to difficulties in conveying the intended emotions in a form the technology can interpret [29]. While programmers use GenAI to streamline their work and enhance efficiency, artists who produce game assets find GenAI-generated art to be below standards and require laborious prompting to achieve acceptable quality [29]. Others have noted that GenAI fails to generate content that works for novel game devices or when tackling complex problems, requiring user oversight (Kawai, 2024). This limitation is particularly significant in information technology contexts, where rapid technological change means that GenAI may encounter knowledge beyond its training data, increasing the likelihood of generating outputs that are structurally plausible but factually incorrect [30].

In engineering and computing education, studies have reported the benefits of teaching software engineering skills by allowing students to build computer games using GenAI. For instance, GenAI has been found to be an effective learning tool for students who solve game development problems [31]. Sadik [31] reported that students in a cohort that used GenAI produced more efficient and functional games than those in the cohort who did not use it.

However, GenAI also has inherent shortcomings that warrant students' critical evaluation of its outputs before use. For example, it may fail to produce properly formatted, functional computer codes [32]. Although it can help students develop quality user stories, it may omit important details [33].

The use of generative AI in other problem solving context

In problem solving contexts beyond game design, research indicates that use of GenAI in education enhances learning outcomes while also introducing unintended challenges [34], [35]. For example, although GenAI facilitates creative idea generation, it can produce an overwhelming number of ideas that are challenging to evaluate [34]. Additionally, it may introduce parameters that were not specified during the idea-generation process, thereby deviating from the intended solution principles [36].

In summary, from both professional and student perspectives, GenAI offers valuable support in problem solving by enhancing work quality and improving time efficiency [10]. However, it can produce inaccurate responses due to its tendency to fabricate facts [30]. Addressing this challenge requires students to critically integrate and validate GenAI output with multiple sources of information, including their prior knowledge [6], as well as to improve their prompting skills [29]. This tension may, in turn, prompt students to engage in epistemic practices. Nevertheless, previous studies focused on what students used GenAI for rather than on the practices they employ to engage with GenAI.

Method

This is a qualitative data analysis in which we analyzed twelve de-identified student projects completed with the support of GenAI in a game design project. We analyzed the projects to answer the question: In what ways do students use GenAI when solving open-ended computer programming problems, and what epistemic practices do they enact through this use?

Participants

The twelve students whose projects were analyzed were all second-year engineering students enrolled in an introductory programming course for engineers, and they were instructed by the same professor.

Data collection

Data was collected from the artifacts generated from completing the project. The artifacts generated included GenAI chat logs and students' written reflections on how they used GenAI. We obtained the de-identified GenAI chat logs and written reflections from the course instructor. A student's project was considered if it contained at least a line of interaction with GenAI and a reflection addressing the student's approach. The data were labelled participant A through participant L.

This project was called "Create a Game" and was assigned to teach students to apply programming language C. Students had the option to work individually or with a partner. Students were required to implement, test, and play a standard input-output-based adventure game.

The instructor outlined at least five decisions students should consider as they developed the game: the size and structure of the adventure board, how movement will work, what players will interact with, kind of character information to keep track of, and the goal of the game. Students were also required to use GenAI to improve understanding of programming concepts, to enhance independently developed algorithms, and to debug code, as well as to write a reflection on how they used it.

Data analysis

The first author read the GenAI chat logs and students' written reflection to develop the initial understanding of the data. He also reviewed the project description the instructor provided to understand the project requirements. He then used the constructivist grounded theory approach [37] to generate qualitative codes and categories from the data (see Figure 1). The first author open-coded the data, capturing the participants' textual interactions with GenAI, and generating initial codes. Simultaneously, he wrote analytical memos describing the ways in which the participants used GenAI. He then grouped the initial codes into focused codes according to their analytical similarities. He categorized the focused codes across all participants into epistemic practices according to the definition of epistemic practices. The second author reviewed the epistemic practices and analytical memos for alignment. The first and second authors then met to discuss any unclear areas. Finally, the first author compared the analytical memos with the categorical codes representing epistemic practices to determine which practices correspond to specific GenAI use cases. Each epistemic practice was evident in at least one of the five identified types of GenAI use cases. For example, epistemic practice of error diagnoses and resolution was applied both when GenAI was used as coding assistant and as a debugging tool.

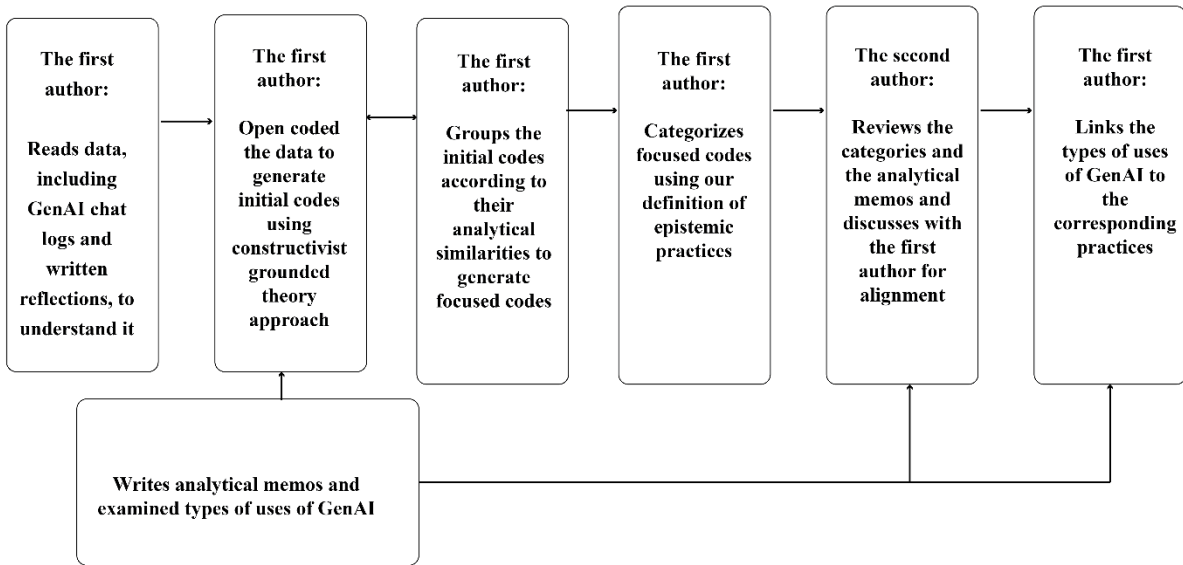


Figure 1 *The Analytic Process*

Results

We identified eleven epistemic practices and five distinct types of GenAI use among students. The GenAI uses and the corresponding epistemic practices are shown in Table 1.

Table 1 *Generative AI Uses and Corresponding Epistemic Practices Among Engineering Students*

Students Uses of Generative AI As:	Students' Epistemic Practices
A Coding Assistant	Initializing the problem space Seeking guidance on function implementation Diagnosing and resolving errors Identifying inaccurate and extraneous information Exploring multiple solution presentations Implicit-explicit prompting strategy
A Debugging Tool	Diagnosing and resolving errors Integrating non-AI sources Reframing prompts to improve output
An Idea Generator	Brainstorming design features and settings Refining and implementing complex design features
An Initial Solution Generator	Initializing the problem space
A Learning Assistant	Seeking efficient use of programming features

In this paper, we describe how students used GenAI as a Coding Assistant and as an Idea Generator along with the corresponding epistemic practices.

GenAI as a coding assistant

Students used GenAI as a coding assistant to improve function writing. As students wrote functions and code segments, they engaged in the epistemic practices of initializing the problem space, seeking guidance on function implementation, diagnosing and resolving errors, and exploring multiple solution presentations.

Students used the epistemic practice of initializing the problem space to understand how to generate codes for the game setup. For this assignment, the game setup included developing the gameboard and rules governing the player. Through a chain of interactions with GenAI, students refined their code to correctly implement the game's structure, objectives, or constraints. For instance, Participant C, through a series of interactions with GenAI, attempted to fix their player movement code: "help create a player 'P' that moves around the board depending on user input using WASD" and "this is my code so far, why is my character not moving?" In this example, Participant C attempted to code a player to meet the project requirements; however, they got stuck. This student correctly identified what they needed to do, which is a critical first step in solving any problem. They used GenAI to help troubleshoot their code to allow them to move beyond the initial stages of the project and work on meeting other project objectives. Using GenAI as a tool to support the epistemic practice of initializing the problem space, allowed the student to get assistance at the early stage of their project to keep making forward progress.

Students most often relied on a prompting pattern we described as the epistemic practice of implicit-explicit prompting strategy. In this epistemic practice, students first employed general instructions then refined them into contextual instructions to enhance their interactions with GenAI. For example, a student might describe a task or problem broadly to GenAI then followed up with more specific instructions. This scenario is illustrated in Participant L's interactions with GenAI. Initially, they input the problematic code, stating, "This is my code so far [...]. Why is my character not moving? They then followed up with a more specific prompt: "when I hit P it says invalid move how do I make it not say that[?]" In this example the students' code was unable to make the player move. When prompting GenAI, they first presented the code, then described the problem, and finally asked for suggestions on how to resolve it.

To integrate the GenAI generated functions into their projects, students first needed to understand how the code worked. They engaged in the epistemic practice of seeking guidance on function implementation to understand how code snippets execute various tasks in the game. When enacting this practice students compared their function to a GenAI generated function and asked GenAI to explain code snippets or sought alternative ways to implement a programming feature. This practice is similar to the strategy of working with a coding buddy, where understanding is co-constructed through guided explanation and iterative clarification. The benefit of GenAI is that it explains the code faster and better than an average coding buddy, as a coding buddy is typically a peer who is also learning to code. This epistemic practice is seen in Participant E's reflection: "We did not understand the compressed version of the for [statement]

first time, so we asked the AI for explanation of the code then we understood it.” In this example, GenAI suggested a code segment containing unfamiliar iteration construct that controlled execution flow. As a result, Participant E was unable to implement the function suggested by GenAI to create a state of uncertainty in the game setting after prompting it with “How to implement fog of war on our board game?” This epistemic practice demonstrates that students are not simply incorporating GenAI-generated code into their workflow without reflection but are examining it to understand its function.

Students examined the efficiency and accuracy of GenAI-generated suggestions, through the epistemic practice of identifying inaccurate and extraneous information. This epistemic practice is illustrated in Participant F’s reflection: “If you are not specific, then AI will provide you with a code with so much detail that we haven’t learned that it would be very hard to understand and read the code.” This example shows that the participant recognized that the code was bulky and they needed more knowledge and effort to interpret it. For this reason, when GenAI-generated code is implemented, runtime errors can occur, or the code may execute differently from what was intended. Students tackle this problem of extraneous data by being very specific in their prompting as illustrated by Participant E’s reflection: “So, for each comment text we gave to AI, we had to be very specific about what code we wanted the AI to do”

Students often struggle to understand GenAI-generated code until multiple examples are presented. Hence, they engaged in the epistemic practice of exploring multiple solution presentations to generate and examine various code snippets. This epistemic practice describes students prompting AI to complete a task in different ways to improve understanding. For example, in their reflection Participant J states, “[GenAI] was able to give me multiple solutions when I asked how to compare strings without case sensitivity.” Likewise, Participant I reflects “Given I didn’t recognize the functions used, I prompted it to use different code.”

Students enacted the epistemic practice of diagnosing and resolving errors when troubleshooting errors in GenAI-generated code. Specifically, students modified GenAI-generated code to meet their task requirements or to make the code run. This epistemic practice is illustrated in Participant B’s reflection: “Although this [GenAI generated code] was a very useful start, it had to be modified to work with what I was going for. I ended up modifying [it].” Similarly, Participant H prompted GenAI with, “can you please redo the last 4 lines it seems that the correct path is disconnected.” Both participants recognized that the GenAI-generated code did not work as intended and sought to modify it. Participant B modified the code on their own, while Participant H relied on GenAI to make revisions. In both cases, the students were able to evaluate the code and determine that there was an issue. With this epistemic practice students are responding to logical or runtime errors in the GenAI-generated code. This practice is important for students to develop in order to responsibly use GenAI. In many ways, this practice is analogous to debugging existing code, a critical skill in coding.

GenAI as an idea generator

Students used GenAI to generate ideas for improving the features of the game. The epistemic practices used when generating ideas included brainstorming design features and settings and refining and implementing complex game features.

In generating ideas for game features, students sampled various game stories, character persona, or game settings by engaging GenAI in the epistemic practice of brainstorming design features and settings. Typically, students provided contextual details about the story or prompted GenAI to generate a narrative centered on a specific character. Through their interactions with GenAI, students sampled multiple story variations, featuring different characters before selecting one to develop further. This epistemic practice is evident in Participant J's interactions with GenAI: "can you give me a short riddle about a werewolf", "can you give me a short riddle about a skeleton", and "can you give me a short riddle about a vampire." It is also seen in Participant H's interactions with GenAI: "can you give me a story for an adventurer going through a dungeon to fight a lich." In these examples, students used GenAI to generate options then examined them to select one that best fits their game. While GenAI produced these ideas, students evaluated and tested them using their own judgement.

These game stories and settings are what make games more engaging and complex but implementing them requires player interaction with the environment. To achieve this, students used the epistemic practice of refining and implementing complex design features. This epistemic practice is manifested when students progressed from developing the structure of the game, such as gameboard and player movement, to implementing its finer details. These finer details included implementing player interaction with the environment and making games challenging to play. These quotes are instances from Participant E's reflection and prompt respectively: "Lastly, the interesting thing was we wanted to make the game more complex, so we asked for some suggestions for making the game complex it gave a lot of cool tips and tricks for making the game complex" and "Make the game harder to play." Participant E used GenAI both to generate ideas for increasing the game's complexity and to support their implementation. This epistemic practice is also evident in Participant K's prompt: "if I do this: void obstacles (char* rock, char* bolder), can the variable rock and bolder be a 2D array or do I have to modify it?" Participant K used GenAI to test the implementation of obstacles to increase the difficulty of the game by asking some predictive questions. If they confirmed the feasibility of an idea, students proceeded to implement it fully in the game, connecting knowledge to problem-solving. This series of prompts illustrates how solution components that can be integrated into existing framework are generated. For example, students can input their project into GenAI and ask it to make certain changes, or they can identify areas that require improvement and seek targeted clarifications on how to implement them. This epistemic practice connects to making finer contextual decisions. It is important because it enables students to evaluate what needs improvement and determine which modifications are most appropriate within the context of their project.

Discussion

Our aim in this study was to uncover the ways students generate, evaluate, and apply knowledge (also referred to as epistemic practices) when using GenAI to solve open-ended computer programming problems. Our investigation is guided by the research questions: In what ways do students use GenAI when solving an open-ended computer programming problem in a second-year introductory programming course, and what epistemic practices do they enact through this use?

We examined how engineering students in an introductory programming class used GenAI to develop games for a class project and the epistemic practices they enacted during these interactions. Students used GenAI in one or more of the following ways: as a coding assistant, debugging tool, initial solution generator, idea generator, and learning assistant. Eleven epistemic practices occurred across the five types of GenAI use cases, with at least one epistemic practice enacted in one use case. More epistemic practices were enacted when GenAI is used as a coding assistant, followed by its use as a debugging tool, then as an idea generator.

The epistemic practices we identified align with categories of creativity and innovation, tools and strategies for problem solving [24], and materializing solutions [23]. When students enacted the epistemic practices of refining and implementing complex design features, and brainstorming design features and settings, they engaged in the creative process, which is consistent with the categorization of these practices as creativity and innovation [24]. GenAI was able to generate several ideas faster than students could on their own. Our work does not speak to whether using GenAI to support brainstorming is “good or bad” for developing creative thinking, but we can say that it was part of students’ creative thinking, much like working with another student. While students can generate numerous ideas or conceptual solutions on their own, testing the feasibility of these ideas may be impractical without the support of GenAI.

Many of the epistemic practices enacted through students’ use of GenAI as a coding assistant, such as initializing the problem space, seeking guidance on function implementation, diagnosing and resolving errors, align with the epistemic practices of tools and strategies for problem solving and the epistemic practice of materializing solution [23]. This use case involves students actively implementing their ideas and hypothetical solutions through writing codes, manipulating code snippets, testing execution, and troubleshooting. In broader engineering practice, this mirrors the process of working with materials, operating tools, and testing prototypes. In the context of this study, these epistemic practices involve students demonstrating extensive use of prompting strategies and actively evaluating and manipulating virtual artifacts.

We suggest that instructors can support students in using appropriate epistemic practices for specific uses of GenAI—such as a coding assistant, initial solution generator, and debugging tool—by documenting and sharing effective practices for each use within their instructional materials. For example, if a student seeks to generate design ideas using GenAI, what constitutes effective engagement in that process? We are not proposing that the epistemic practices identified in this study are inherently effective or ineffective. Rather, we suggest that instructors examine

these practices to better understand how students are approaching GenAI use, and to consider whether these approaches are appropriate and how they might be refined or supported.

Deeper insights could have been obtained through interviews with the participants, which we consider a major limitation of this study. We hope that future research will benefit from using our findings to guide the design of interview protocols. For example, during interviews, students can be asked: “Can you describe how you used GenAI to generate code. What kind of errors, if any, did you find in the GenAI-generated code, and how did you resolve them? Our identified epistemic practices will be useful for interpreting students’ responses.

Conclusion

In this study, through the analysis of students’ project artifacts, we identified eleven epistemic practices demonstrated in the use of GenAI to support problem solving. These epistemic practices correspond to specific GenAI use cases, which in turn are determined by their problem-solving goal. Our analysis revealed five primary ways students employed GenAI: as a coding assistant, debugging tool, idea generator, initial solution generator, and learning assistant. This pattern of GenAI use reflects students’ awareness of its multiple functionalities, as uses that were more central to students’ problem solving with GenAI involved a wider range of epistemic practices. The same epistemic practice can support different forms of reasoning depending on how students use GenAI in their problem-solving activities. For example, when using GenAI as a coding assistant, students enacted the epistemic practice of initializing the problem space while generating code for the game setup. In contrast, when using GenAI as an initial solution generator, they demonstrated the same epistemic practice to gain an understanding of how to begin solving the problem.

References

- [1] N. E. Cagiltay, “Teaching software engineering by means of computer-game development: Challenges and opportunities,” *Br. J. Educ. Technol.*, vol. 38, no. 3, pp. 405–415, May 2007, doi: 10.1111/j.1467-8535.2007.00705.x.
- [2] K. Claypool and M. Claypool, “Teaching software engineering through game design,” *ACM SIGCSE Bull.*, vol. 37, no. 3, pp. 123–127, Sep. 2005, doi: 10.1145/1151954.1067482.
- [3] S. Kurkovsky, “Engaging students through mobile game development,” *ACM SIGCSE Bull.*, vol. 41, no. 1, pp. 44–48, Mar. 2009, doi: 10.1145/1539024.1508881.
- [4] Y. L. Eow, W. Z. B. W. Ali, R. Bt. Mahmud, and R. Baki, “Computer games development and appreciative learning approach in enhancing students’ creative perception,” *Comput. Educ.*, vol. 54, no. 1, pp. 146–161, Jan. 2010, doi: 10.1016/j.compedu.2009.07.019.
- [5] F. French, D. Levi, C. Maczo, A. Simonaityte, S. Triantafyllidis, and G. Varda, “Creative Use of OpenAI in Education: Case Studies from Game Development,” *Multimodal Technol. Interact.*, vol. 7, no. 8, p. 81, Aug. 2023, doi: 10.3390/mti7080081.
- [6] H. E. (Grace) Park, “The double-edged sword of generative artificial intelligence in digitalization: An affordances and constraints perspective,” *Psychol. Mark.*, vol. 41, no. 11, pp. 2924–2941, Nov. 2024, doi: 10.1002/mar.22094.

- [7] G. Kelly, "Inquiry, activity and epistemic practice," in *Teaching scientific inquiry*, Brill, 2008, pp. 99–117.
- [8] G. J. Kelly, "Methodological considerations for the study of epistemic cognition in practice," in *Handbook of epistemic cognition*, Routledge, 2016, pp. 393–408.
- [9] S. A. Alharthi, "Generative AI in Game Design: Enhancing Creativity or Constraining Innovation?," *J. Intell.*, vol. 13, no. 6, p. 60, May 2025, doi: 10.3390/jintelligence13060060.
- [10] J. Lee, S.-Y. Eom, and J. Lee, "Empowering game designers with generative AI," *IADIS Int. J. Comput. Sci. Inf. Syst.*, vol. 18, no. 2, pp. 213–230, 2023.
- [11] I. J. P. Colado, V. M. P. Colado, A. C. Morata, R. S. C. Píriz, and B. F. Manjón, "Using New AI-Driven Techniques to Ease Serious Games Authoring," in *2023 IEEE Frontiers in Education Conference (FIE)*, College Station, TX, USA: IEEE, Oct. 2023, pp. 1–9. doi: 10.1109/FIE58773.2023.10343021.
- [12] B. K. Hofer, "Epistemic cognition as a psychological construct: Advancements and challenges," in *Handbook of epistemic cognition*, Routledge, 2016, pp. 19–38.
- [13] K. S. Kitchner, "Cognition, metacognition, and epistemic cognition: A three-level model of cognitive processing," *Hum. Dev.*, vol. 26, no. 4, pp. 222–232, 1983.
- [14] P. M. King and K. S. Kitchener, *Developing Reflective Judgment: Understanding and Promoting Intellectual Growth and Critical Thinking in Adolescents and Adults. Jossey-Bass Higher and Adult Education Series and Jossey-Bass Social and Behavioral Science Series*. ERIC, 1994.
- [15] D. Kuhn, *The skills of argument*. Cambridge University Press, 1991.
- [16] M. B. B. Magolda, *Knowing and reasoning in college: Gender-related patterns in students' intellectual development*. Jossey-Bass, 1992.
- [17] M. Schommer, "Effects of beliefs about the nature of knowledge on comprehension.," *J. Educ. Psychol.*, vol. 82, no. 3, p. 498, 1990.
- [18] B. K. Hofer and P. R. Pintrich, "The development of epistemological theories: Beliefs about knowledge and knowing and their relation to learning," *Rev. Educ. Res.*, vol. 67, no. 1, pp. 88–140, 1997.
- [19] L. Louca, A. Elby, D. Hammer, and T. Kagey, "Epistemological resources: Applying a new epistemological framework to science instruction," *Educ. Psychol.*, vol. 39, no. 1, pp. 57–68, 2004.
- [20] J. Tenenberg and D. Chinn, "Epistemic practices in conceptions of computer science," *Comput. Sci. Educ.*, vol. 35, no. 1, pp. 190–211, Jan. 2025, doi: 10.1080/08993408.2024.2381398.
- [21] C. Chinn and W. Sandoval, "Epistemic cognition and epistemic development," in *International handbook of the learning sciences*, Routledge, 2018, pp. 24–33.
- [22] C. A. Chinn and R. W. Rinehart, "Epistemic cognition and philosophy: Developing a new framework for epistemic cognition," in *Handbook of epistemic cognition*, Routledge, 2016, pp. 460–478.
- [23] C. Simarro and D. Couso, "Engineering practices as a framework for STEM education: a proposal based on epistemic nuances," *Int. J. STEM Educ.*, vol. 8, no. 1, p. 53, Dec. 2021, doi: 10.1186/s40594-021-00310-2.
- [24] C. M. Cunningham and G. J. Kelly, "Epistemic Practices of Engineering for Education," *Sci. Educ.*, vol. 101, no. 3, pp. 486–505, May 2017, doi: 10.1002/sc.21271.

- [25] J. Lee, S.-Y. Eom, and J. Lee, "Empowering game designers with generative AI," *IADIS Int. J. Comput. Sci. Inf. Syst.*, vol. 18, no. 2, pp. 213–230, 2023.
- [26] R. Panchanadikar and G. Freeman, "'I'm a Solo Developer but AI is My New Ill-Informed Co-Worker': Envisioning and Designing Generative AI to Support Indie Game Development," *Proc. ACM Hum.-Comput. Interact.*, vol. 8, no. CHI PLAY, pp. 1–26, Oct. 2024, doi: 10.1145/3677082.
- [27] Y. Sun, Z. Li, K. Fang, C. H. Lee, and A. Asadipour, "Language as Reality: A Co-Creative Storytelling Game Experience in 1001 Nights Using Generative AI," *Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entertain.*, vol. 19, no. 1, pp. 425–434, Oct. 2023, doi: 10.1609/aiide.v19i1.27539.
- [28] A. Begemann and J. Hutson, "Empirical insights into AI-assisted game development: A case study on the integration of generative AI tools in creative pipelines," *Metaverse*, vol. 5, no. 2, p. 2568, Jul. 2024, doi: 10.54517/m.v5i2.2568.
- [29] J. D. Boucher, G. Smith, and Y. D. Telliel, "Is Resistance Futile?: Early Career Game Developers, Generative AI, and Ethical Skepticism," in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, Honolulu HI USA: ACM, May 2024, pp. 1–13. doi: 10.1145/3613904.3641889.
- [30] D. Anh-Hoang, V. Tran, and L.-M. Nguyen, "Survey and analysis of hallucinations in large language models: attribution to prompting strategies or model behavior," *Front. Artif. Intell.*, vol. 8, p. 1622292, 2025.
- [31] O. Sadik, "Using Generative AI to Design and Develop Educational Games in Engineering Education," *Int. J. Des. Learn.*, vol. 16, no. 2, pp. 182–193, Dec. 2025, doi: 10.14434/ijdl.v16i2.42059.
- [32] B. Puryear and G. Sprint, "Github copilot in the classroom: learning to code with AI assistance," *J. Comput. Sci. Coll.*, vol. 38, no. 1, pp. 37–47, 2022.
- [33] A. Brockenbrough and D. Salinas, "Using Generative AI to Create User Stories in the Software Engineering Classroom," in *2024 36th International Conference on Software Engineering Education and Training (CSEET&T)*, Jul. 2024, pp. 1–5. doi: 10.1109/CSEET62301.2024.10662994.
- [34] R. Baltà-Salvador, I. El-Madafri, E. Brasó-Vives, and M. Peña, "Empowering Engineering Students Through Artificial Intelligence (AI): Blended Human–AI Creative Ideation Processes With ChatGPT," *Comput. Appl. Eng. Educ.*, vol. 33, no. 1, p. e22817, Jan. 2025, doi: 10.1002/cae.22817.
- [35] I. Khan, I. Chohan, and S. I. Malik, "Leveraging ChatGPT-4 for Enhanced Education: Personalized Problem-Solving and Consistent Learning," in *2024 2nd International Conference on Computing and Data Analytics (ICCD A)*, Shinas, Oman: IEEE, Nov. 2024, pp. 1–6. doi: 10.1109/ICCD A64887.2024.10867340.
- [36] P. Livotov and Mas'udah, "Challenges in Inventive Design Problem Solving with Generative AI: Interactive Problem Definition, Multi-directional Prompting, and Concept Development," in *World Conference of AI-Powered Innovation and Inventive Design*, vol. 735, D. Cavallucci, S. Brad, and P. Livotov, Eds., in IFIP Advances in Information and Communication Technology, vol. 735. , Cham: Springer Nature Switzerland, 2025, pp. 205–226. doi: 10.1007/978-3-031-75919-2_13.
- [37] K. Charmaz, *Constructing grounded theory: A practical guide through qualitative analysis*. sage, 2006.